Project Planning & Management

E-Commerce Data Integration & Analytics Pipeline

Digital Egypt Pioneers Initiative (DEPI) - Data Engineering Track

Delivery date: 16 October 2026

1. Project Proposal

1.1 Overview

An e-commerce company gets its data from different places: product data, customer data, order data and transaction data. This project builds an automated data pipeline that collects this data, cleans it, stores it in a relational database, and prepares a data model that is ready for analysis.

1.2 Problem Statement

The company receives data from CSV files, an external API and a product website. The data has duplicates, missing values, inconsistent formats and records that change over time. Analysts need one reliable central dataset to build sales reports.

1.3 Objectives

- Collect data from many sources: CSV/JSON files and a REST API.
- Check, clean and standardize the data (missing values, duplicates, data types and formats).
- Store the data in a relational database with a star schema.
- Run the whole process automatically with Apache Airflow.
- Make the data ready for sales reporting using SQL views.

1.4 Scope

What the project includes:

- Get product data from a REST API.
- Read customer and order data from CSV/JSON files.
- Validate the incoming data.
- Handle missing values and duplicates.
- Standardize data types and formats.
- Join and transform the data with Pandas.
- Design a relational database.
- Create a star schema: Fact Sales, Dim Customer, Dim Product and Dim Date.
- Load the data incrementally (append-based).
- Create SQL views for common analysis questions.
- Add indexes to the fields that are queried often.
- Build an Airflow DAG: Extract, Validate, Transform, Load, Quality Check.
- Add logging, retry and error handling.

1.5 Expected Deliverables

All deliverables are uploaded to one GitHub repository:

- Python ETL modules
- Database schema and ERD
- SQL scripts
- Airflow DAG
- Sample input datasets
- Data-quality checks
- Pipeline documentation
- Architecture diagram
- Screenshots of successful Airflow runs
- README file that explains the full architecture

2. Project Plan

2.1 Timeline (Gantt Chart)

The work is split into phases. A colored cell means the phase is active in that week. The weeks start on Monday.

Phase	Dates	5 Oct	12 Oct	19 Oct	26 Oct	2 Nov	9 Nov	16 Nov	23 Nov	30 Nov
Reading and summarizing the project	7 Oct									
Planning, Literature Review, Requirements	7 - 16 Oct									
System Analysis & Design	17 Oct - 6 Nov									
Implementation (build the pipeline)	7 - 30 Nov									
Final Presentation, Testing & Reports	1 - 4 Dec									

Note: the phase dates inside each deadline are our own plan. The official deadlines are in the milestones table below.

2.2 Milestones and Deliverables

Milestone	Deadline	Deliverable
M1 - Planning	16 Oct 2026	Project Planning & Management document, Literature Review, Requirements Gathering
M2 - Design	6 Nov 2026	System Analysis & Design: architecture, ERD, database schema, data flow diagrams
M3 - Implementation	30 Nov 2026	Working pipeline: source code, Airflow DAG, database, README, GitHub repository
M4 - Final	4 Dec 2026	Testing, reports, user manual, final presentation and infographic

2.3 Resource Allocation

Team: 6 members (see Section 3). The tools we use are:

Resource	Use in the project
Python 3.11	Writing the ETL code
Pandas	Cleaning, joining and transforming data
requests library	Getting product data from the REST API
SQLite	Relational database for the star schema
Apache Airflow (on WSL Ubuntu)	Running the pipeline automatically
GitHub	Storing code and all documents

3. Task Assignment & Roles

Each member is responsible for one stage of the pipeline.

#	Name	Role	Responsibility
1	نوران حمدي شوقي راغب	Database and Load	Designs the tables and star schema and loads the data into the database
2	حسانين رمضان محمد صبره	Validate	Checks the incoming data (missing values, data types) before processing
3	عبدالرحمن محمد أحمد صبري	SQL Views, Indexes and Quality Check	Creates the analytical views and indexes and checks data quality after loading
4	هدي نصر اسماعيل صالح	Transform	Cleans the data, removes duplicates and standardizes formats using Pandas
5	احمد محمد ربيع عبد الغفور	Airflow DAG, Logging and Retry	Connects all pipeline stages and schedules them to run automatically
6	عامر محمد صبري صابر احمد	Extract	Reads the CSV/JSON files and gets the products from the API

4. Risk Assessment & Mitigation Plan

Risk	Level	How we reduce it
Airflow does not run directly on Windows	High	Run Airflow inside WSL Ubuntu, and test the setup early.
The external API is down or changes its response	Medium	Keep a saved copy of a sample API response to test with; add retry for API calls.
Bad data breaks the pipeline (missing values, wrong types)	High	Validate the data first, handle errors, and run a Quality Check at the end.
A task fails in the middle of a run	Medium	Use Airflow retry and write clear logs to find the problem fast.
Team members have different skill levels	Medium	Give simple roles, help each other, and review the work together.
Missing a deadline	Medium	Follow the milestones and check progress every week.
Conflicts when many people edit the same files on GitHub	Low	Each member works on separate files, and we upload often.

5. KPIs (Key Performance Indicators)

KPI	Target
Successful pipeline runs in Airflow	All 5 tasks finish green (Extract, Validate, Transform, Load, Quality Check)
Duplicate records in the final tables	0
Missing values in key columns after cleaning	0
Valid records loaded into the database	100% of the records that passed validation
Data-quality checks passed	100%
Failed tasks that are retried and logged	Every failed task is retried and recorded in the log
Deliverables submitted on time	All 4 milestones by their deadlines

Literature Review

E-Commerce Data Integration & Analytics Pipeline

Digital Egypt Pioneers Initiative (DEPI) - Data Engineering Track

Delivery date: 16 October 2026

1. Purpose

This document looks at solutions and projects that are similar to ours. For each one we write what it is, its advantages and its disadvantages. Then we compare them with our project and explain what we learned from them. The sources are listed at the end.

2. How We Chose the Solutions

Our project collects e-commerce data from files and an API, cleans it, loads it into a star schema and runs automatically. So we looked for three kinds of similar solutions:

- Cloud data integration services: Azure Data Factory and AWS Glue. Companies use them, and Azure is part of our course.
- An open-source data movement tool: Airbyte.
- Open-source projects on GitHub that build a data pipeline for e-commerce or retail data (two projects).

We read the official documentation of the tools and the README file of each GitHub project. Our notes are based on what these pages say.

3. Similar Solutions

3.1 Azure Data Factory [1]

A managed cloud service from Microsoft for ETL, ELT and data integration. A pipeline is made of activities, and triggers start it.

Advantages

- It has many ready-made connectors for cloud and on-premises data sources.
- Pipelines can start on a schedule or when an event happens, and they are monitored from the Azure portal.
- Visual data flows run on Spark, and the user does not manage the clusters.
- It supports CI/CD with Azure DevOps and GitHub.

Disadvantages

- It needs an Azure subscription and runs in the cloud.
- It has many concepts to learn (linked services, datasets, integration runtimes), which is a lot for a small project.
- Microsoft says Data Factory in Microsoft Fabric is the next generation and recommends it for new users, so the tool is changing.

3.2 AWS Glue [2]

A serverless data integration service from Amazon. It helps to find, prepare, move and combine data from many sources, and to build ETL pipelines for data lakes.

Advantages

- It is serverless, so there are no servers to manage.
- Crawlers find the schema of the data and save it in the Data Catalog.
- A visual editor (Glue Studio) creates the ETL jobs, and jobs can run on a schedule, on demand or after an event.
- It has a built-in feature (FindMatches) to find duplicate records.
- It uses pay-as-you-go billing.

Disadvantages

- It needs an AWS account and the setup of IAM permissions.
- Pay-as-you-go billing means the cost grows with the use.
- It is built to work with other AWS services (such as S3, Athena and Redshift).

3.3 Airbyte [3]

An open-source data replication platform. It copies data from many sources to data warehouses, data lakes and databases.

Advantages

- It is open source.
- It has more than 600 sources and destinations (connectors).
- It can be used as a cloud service or installed and managed by the user.

Disadvantages

- Its documentation page describes moving (replicating) data. Cleaning and joining the data needs another tool; this is our understanding, because the page does not mention transformations.
- The cloud version starts with a 14-day free trial, and the self-managed version needs to be installed and maintained.

3.4 Retail Data Warehousing Project, GitHub: marccassa [4]

A small project that turns a CSV file of sales into a data warehouse. It uses Python and Pandas to clean the data, SQLAlchemy to write it into SQLite, a star schema (one fact table and three dimension tables), SQL analysis and Matplotlib charts.

Advantages

- It uses tools that are very close to ours: Python, Pandas, SQLite and a star schema.
- It is simple: it runs with two Python scripts.
- It includes SQL analysis of the results.

Disadvantages

- It runs by hand. The README does not mention a scheduler such as Airflow.
- The README does not mention validation, retry, logging or incremental loading.
- As described, it has one CSV source only.

3.5 EcomBI, GitHub: Seakty [5]

An end-to-end e-commerce project. Airflow gets users, products and carts from the DummyJSON API and loads them into PostgreSQL. dbt builds the staging and mart tables. A Streamlit dashboard shows the results and product recommendations, and a FastAPI service gives access to the data. All services run with Docker Compose.

Advantages

- It uses the same API (DummyJSON) and the same orchestration tool (Airflow) as our project.
- It is a complete solution: dashboard, recommendations and a REST API.
- It keeps the transformations in layers with dbt (staging and marts).

Disadvantages

- It is heavy: it needs Docker Desktop, at least 4 GB of RAM (8 GB recommended) and about 10 GB of free disk space.
- The start takes 1 to 2 minutes, and port conflicts can stop the services.
- It depends on the DummyJSON API being available.
- It uses many tools (dbt, FastAPI, Streamlit, scikit-learn, Docker), which is hard for a beginner.

4. Comparison With Our Project

Solution	Type	What you need	Cleaning and scheduling	Fit for our case
Azure Data Factory	Cloud service	An Azure subscription	Data flows on Spark, and triggers	Too big; needs a cloud account
AWS Glue	Cloud service	An AWS account and IAM setup	Visual ETL jobs, schedules and events	Too big; pay per use
Airbyte	Open-source tool	Installation, or a cloud account	Moves the data; cleaning needs another tool	Good only for moving data
Retail Data Warehousing (GitHub)	Student project	Python, Pandas and SQLite	Cleaning in Pandas; runs by hand	Close in tools, but no scheduler
EcomBI (GitHub)	Full project	Docker, PostgreSQL, 8 GB RAM recommended	Airflow and dbt	Close in idea, but heavy
Our project	**Student project**	**Python, Pandas, SQLite and Airflow on WSL (all free)**	**Validate, Transform and Quality Check tasks; Airflow @daily with retry and logs**	**Small and free; a full run takes about 30 seconds**

5. What We Learned From Them

- Data Factory and Glue: a pipeline is a chain of steps with a schedule, monitoring and retry. Our Airflow DAG has five tasks, a daily schedule, retries and logs.
- Glue: removing duplicates and checking the quality of the data is part of the pipeline. We have the Validate and Quality Check tasks, and Transform removes duplicate rows.
- Airbyte: every source should be handled as a separate step. Our Extract module reads the three sources (CSV, JSON and the API) one by one.
- The Retail project: a star schema with one fact table and dimension tables is a good model for sales analysis. We use fact_sales with dim_customer, dim_product and dim_date.
- EcomBI: Airflow with the DummyJSON API works well for a learning project. We use the same idea, but without Docker, to keep the setup simple.

6. How Our Project Is Different

What is different:

- It is smaller and simpler: no Docker, no cloud account and no cost. It runs on a normal laptop.
- Validation and the data-quality check are separate Airflow tasks, loading is incremental (only new orders are added), and every step writes logs. A failed task is retried, except Validate and Quality Check, because a retry cannot fix bad data.

What the bigger solutions do better:

- They have many more connectors (Airbyte, Glue) and can scale to very large data (Data Factory, Glue).
- EcomBI has a dashboard, product recommendations and a REST API; our project has none of these.
- Our loading is append-only (a changed old record is not updated) and Transform understands only three date formats. These are limits that we can improve later.

7. Conclusion

The cloud tools and Airbyte show how data pipelines are built in real companies, but they need accounts, setup or money. The two GitHub projects are closer to ours: one is simple but runs by hand, the other is complete but heavy. Our project takes the useful ideas from all of them (pipeline steps, a schedule, retry, data quality and a star schema) and keeps them small, free and easy to run and explain.

8. References

Accessed on 26 October 2026.

[1] Microsoft. Introduction to Azure Data Factory. Microsoft Learn. https://learn.microsoft.com/en-us/azure/data-factory/introduction

[2] Amazon Web Services. What is AWS Glue? AWS Glue Developer Guide. https://docs.aws.amazon.com/glue/latest/dg/what-is-glue.html

[3] Airbyte. Airbyte Documentation. https://docs.airbyte.com/

[4] marccassa. Retail Data Warehousing Project Python SQL. GitHub. https://github.com/marccassa/Retail-Data-Warehousing-Project-Python-SQL

[5] Seakty. EcomBI. GitHub. https://github.com/Seakty/EcomBI

Requirements Gathering

E-Commerce Data Integration & Analytics Pipeline

Digital Egypt Pioneers Initiative (DEPI) - Data Engineering Track

Delivery date: 16 October 2026

1. Stakeholder Analysis

A stakeholder is any person or group that uses the system or is affected by it. These are the stakeholders of our pipeline and what each one needs:

Stakeholder	What they need	How they use the project
Data analysts	One reliable, clean and central dataset with no duplicates or missing values	Query the star schema and the SQL views to build sales reports
Business managers	Correct sales numbers to take decisions	Read the reports that come from the analysts' queries
Data engineering team (us)	A pipeline that runs automatically, is easy to understand and shows clear logs when something fails	Build, run and maintain the pipeline in Airflow
E-commerce company	Data from many sources combined in one place	Owns the data sources (files, API) and the final database
DEPI instructor and evaluators	All required deliverables, documented and uploaded to GitHub	Review and grade the project

2. User Stories and Use Cases

2.1 User Stories

ID	As a...	I want to...	So that...
US-1	Data analyst	get customer, order and product data in one database	I do not collect the data by hand from different sources
US-2	Data analyst	have duplicates and missing values handled	my reports are correct
US-3	Data analyst	use ready SQL views	I can answer common sales questions quickly
US-4	Business manager	see correct sales figures	I can take good decisions
US-5	Data engineer	run the whole pipeline automatically in Airflow	I do not run each step manually
US-6	Data engineer	see logs and retries when a task fails	I can find and fix the problem fast
US-7	Data engineer	have a quality check at the end of each run	I know the loaded data is correct
US-8	Instructor	find the code, documents and screenshots on GitHub	I can evaluate the project

2.2 Use Cases

Use case	Actor	Main steps	Result
UC-1: Run the pipeline	Data engineer	1. Open Airflow. 2. Start (trigger) the DAG. 3. Airflow runs Extract, Validate, Transform, Load and Quality Check.	Clean data is loaded into the database
UC-2: Check a pipeline run	Data engineer	1. Open the DAG run in Airflow. 2. See the status of each task. 3. Open the log of a failed task.	The engineer knows if the run succeeded or why it failed
UC-3: Analyze sales	Data analyst	1. Open the database. 2. Run a SQL view or query on the star schema.	Sales results for reports
UC-4: Review data quality	Data engineer	1. Open the Quality Check task result. 2. Read the checks that passed or failed.	Confirmation that the final data is correct

3. Functional Requirements

These are the features the system must have:

ID	Requirement
FR-1	The system shall get product data from a REST API.
FR-2	The system shall read customer and order data from CSV/JSON files.
FR-3	The system shall validate the incoming data (for example, required fields and data types).
FR-4	The system shall handle missing values.
FR-5	The system shall remove duplicate records.
FR-6	The system shall standardize data types and formats.
FR-7	The system shall join and transform the data using Pandas.
FR-8	The system shall store the data in a relational database with a star schema: Fact Sales, Dim Customer, Dim Product and Dim Date.
FR-9	The system shall load the data incrementally (append-based).
FR-10	The system shall provide SQL views for common analysis queries.
FR-11	The system shall have indexes on the fields that are queried often.
FR-12	The system shall run a data-quality check after loading.
FR-13	The system shall run through an Airflow DAG with the tasks: Extract, Validate, Transform, Load, Quality Check.
FR-14	The system shall write logs and retry a failed task.

4. Non-functional Requirements

These describe how well the system should work:

ID	Type	Requirement
NFR-1	Reliability	A bad record or a failed task must not stop the whole pipeline without a clear error; failed tasks are retried.
NFR-2	Data quality	The final tables must have no duplicates and no missing values in the key columns.
NFR-3	Performance	A full run on the sample data finishes in less than 1 minute (measured about 30 seconds in Airflow).
NFR-4	Usability	The README explains how to install and run the project step by step in simple words.
NFR-5	Maintainability	The code is split into small modules (extract, validate, transform, load, quality check) with clear names and comments.
NFR-6	Observability	Every step writes clear logs, and the run status is visible in the Airflow interface.
NFR-7	Portability	The project runs with Python 3.11 and Airflow on Ubuntu (WSL), using only free tools.
NFR-8	Security	No passwords or secret keys are written inside the code.